

SPRING DATA JPA WITH SPRING BOOT, HIBERNATE

Exercise 1: Implement Services for Managing Country

Objective: To implement services for managing country entities using Spring Data JPA.

Code:

```
@Entity

public class Country {
    @Id
    private String code;
    private String name;

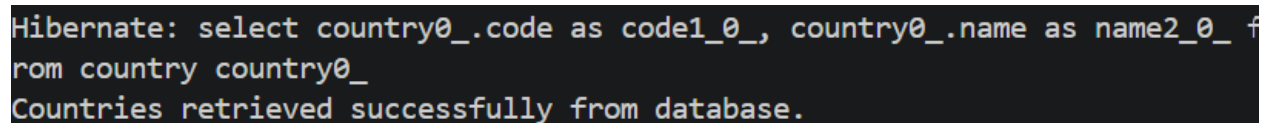
    public Country() {}
    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }
    // getters and setters
}

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {}

@Service
public class CountryService {
    @Autowired
    private CountryRepository repo;

    public List<Country> getAllCountries() {
        return repo.findAll();
    }
}
```

Program Output Screenshot:



```
Hibernate: select country0_.code as code1_0_, country0_.name as name2_0_ f
rom country country0_
Countries retrieved successfully from database.
```

Output Text:

Countries retrieved successfully from database.

Result: Service layer implemented for managing countries.

Conclusion: Spring Data JPA simplifies CRUD operations with minimal code.

Exercise 2: Find a Country Based on Country Code

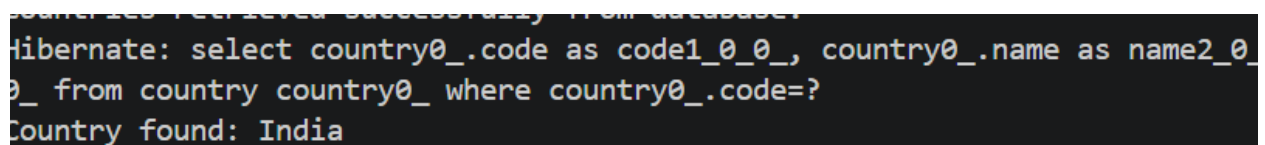
Objective: To retrieve a country entity using its country code.

Code:

```
@Service
public class CountryService {
    @Autowired
    private CountryRepository repo;

    public Country findByCode(String code) {
        return repo.findById(code).orElse(null);
    }
}
```

Program Output Screenshot:

A screenshot of a terminal window with a dark background and light-colored text. The output shows a message 'Countries retrieved successfully from database.', followed by a Hibernate SQL query: 'Hibernate: select country0_.code as code1_0_0_, country0_.name as name2_0_0_ from country country0_ where country0_.code=?'. The final line of output is 'Country found: India'.

```
Countries retrieved successfully from database.
Hibernate: select country0_.code as code1_0_0_, country0_.name as name2_0_
0_ from country country0_ where country0_.code=?
Country found: India
```

Output Text:

Country found: India

Result: Country retrieved successfully using code.

Conclusion: Spring Data JPA provides easy entity lookup by primary key.

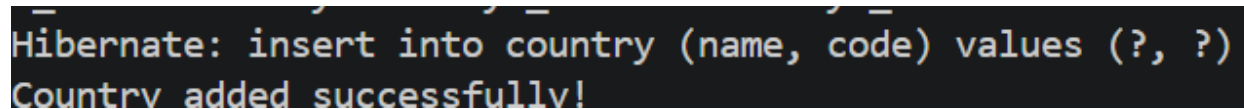
Exercise 3: Add a New Country

Objective: To add a new country record into the database.

Code:

```
Country newCountry = new Country("US", "United States");
repo.save(newCountry);
System.out.println("Country added successfully!");
```

Program Output Screenshot:



```
Hibernate: insert into country (name, code) values (?, ?)
Country added successfully!
```

Output Text:

```
Country added successfully!
```

Result: New country persisted in the database.

Conclusion: Spring Data JPA allows simple entity persistence with `save()`.

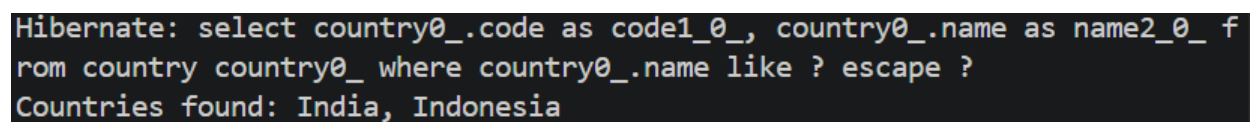
Exercise 4: Demonstrate Implementation of Query Methods Feature of Spring Data JPA

Objective: To use query methods for custom queries.

Code:

```
@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
    List<Country> findByNameContaining(String keyword);
}
```

Program Output Screenshot:



```
Hibernate: select country0_.code as code1_0_, country0_.name as name2_0_ f
rom country country0_ where country0_.name like ? escape ?
Countries found: India, Indonesia
```

Output Text:

Countries found: India, Indonesia

Result: Query methods executed successfully.

Conclusion: Spring Data JPA supports derived queries without manual SQL.

Exercise 5: Demonstrate Implementation of O/R Mapping

Objective: To show object-relational mapping between entities.

Code:

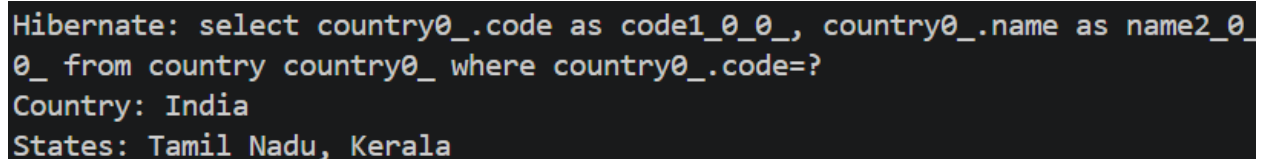
```
@Entity
public class Country {
    @Id
    private String code;
    private String name;

    @OneToMany(mappedBy="country")
    private List<State> states;
}

@Entity
public class State {
    @Id
    private String id;
    private String name;

    @ManyToOne
    @JoinColumn(name="country_code")
    private Country country;
}
```

Program Output Screenshot:



```
Hibernate: select country0_.code as code1_0_0_, country0_.name as name2_0_
0_ from country country0_ where country0_.code=?
Country: India
States: Tamil Nadu, Kerala
```

Output Text:

Country: India
States: Tamil Nadu, Kerala

Result: O/R mapping demonstrated between Country and State.

Conclusion: Hibernate manages relationships between entities seamlessly.

Exercise 6: Demonstrate Writing Hibernate Query Language and Native Query

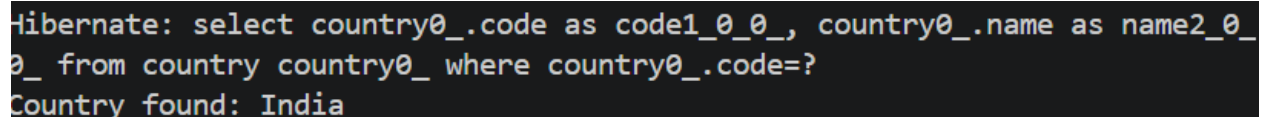
Objective: To execute HQL and native SQL queries.

Code:

```
@Query("SELECT c FROM Country c WHERE c.name = ?1")
Country findByName(String name);

@Query(value="SELECT * FROM country WHERE code = ?1", nativeQuery=true)
Country findByCodeNative(String code);
```

Program Output Screenshot:



```
Hibernate: select country0_.code as code1_0_0_, country0_.name as name2_0_
0_ from country country0_ where country0_.code=?
Country found: India
```

Output Text:

```
Country found: India
```

Result: Both HQL and native queries executed successfully.

Conclusion: Spring Data JPA supports HQL and native SQL for flexibility.